

07.3: Serialization Formats



UMD DATA605 - Big Data Systems

07.3: Serialization Formats

Instructor: Dr. GP Saggese, gsaggese@umd.edu

1 / 9

2 / 9: Serialization Formats

Serialization Formats

- **Programs need to send data to each other**
 - Through network, disk
 - Examples:
 - Remote Procedure Calls (RPCs)
 - Data storage and retrieval
 - Technologies based on schemas:
 - JSON
 - YAML
 - Protocol Buffers
 - Python Pickle
- **Serialization formats are data models**
 - Define how data is structured and stored
 - Examples:
 - JSON: Human-readable, used for web APIs
 - YAML: Human-readable, used for configuration files
 - Protocol Buffers: Efficient, used by Google for internal APIs
 - Python Pickle: Python-specific, used for object serialization

2 / 9

- **Serialization Formats**
 - *Programs need to send data to each other*
 - When different programs or systems need to communicate, they often need to exchange data. This can happen over a network or by writing data to a disk. For instance, when you use a web service, your computer sends data to a server, which processes it and sends back a response.
 - **Examples:**
 - **Remote Procedure Calls (RPCs):** This is a way for a program to execute a procedure (or function) on a different address space, often on another computer. Serialization is crucial here because it allows the data to be packaged and sent across the network.
 - **Data storage and retrieval:** When data is stored in a database or a file, it needs to be serialized so it can be saved and later retrieved in the same format.
 - **Technologies based on schemas:**
 - **JSON:** A lightweight data interchange format that's easy for humans to read and write, and easy for machines to parse and generate. It's widely used in web APIs.
 - **YAML:** Similar to JSON but more human-readable, often used for configuration files due to its simplicity and readability.
 - **Protocol Buffers:** Developed by Google, this is a method for serializing structured data. It's more efficient than JSON or YAML, especially for internal APIs where performance is critical.
 - **Python Pickle:** A Python-specific serialization format used to convert Python objects into a byte stream, making it easy to save and load Python objects.

- **Serialization formats are data models**
 - Serialization formats define the structure and storage of data. They act as a blueprint for how data is organized, ensuring that it can be correctly interpreted when read back.
- **Examples:**
 - **JSON:** Known for its simplicity and readability, JSON is often used in web applications to transmit data between a server and a client.
 - **YAML:** While similar to JSON, YAML is more user-friendly for configuration files due to its clean syntax.
 - **Protocol Buffers:** Offers a compact binary format, making it faster and more efficient for data exchange, especially in high-performance environments.
 - **Python Pickle:** Specifically designed for Python, it allows for the serialization of complex Python objects, making it useful for saving program states or sharing data between Python applications.

3 / 9: Comma Separated Values (CSV)

Comma Separated Values (CSV)

- **CSV stores data row-wise**
 - As text without schema
 - Each line is a data record
 - Fields separated by commas
- **Pros**
 - Very portable
 - Text format
 - Supported by all tools
 - Human-friendly
- **Cons**
 - Large footprint
 - Requires compression
 - Parsing is CPU intensive
 - No easy random access
 - Can't read only a subset of columns
 - No schema/types
 - Annotate CSV with schema
 - Mainly read-only, hard to modify

Year	Make	Model	Description	Price
1997	Ford	E350	ac, abs, moon	3000.00
1999	Chevy	Venture "Extended Edition"		4900.00
1999	Chevy	Venture "Extended Edition, Very Large"		5000.00
1996	Jeep	Grand Cherokee	MUST SELL! air, moon roof, loaded	4799.00

```

Year,Make,Model,Description,Price
1997,Ford,E350,"ac, abs, moon",3000.00
1999,Chevy,"Venture ""Extended Edition""", "",4900.00
1999,Chevy,"Venture ""Extended Edition, Very Large""", "",5000.00
1996,Jeep,Grand Cherokee,"MUST SELL!
air, moon roof, loaded",4799.00

```

3 / 9

- **Comma Separated Values (CSV)**
 - CSV files store data in a simple, row-wise format. Each line represents a data record, and fields within a record are separated by commas. This makes CSV files easy to read and write using basic text editors.
- **Pros**
 - *Very portable:* CSV is a text format, which means it can be opened and edited by almost any software, from simple text editors to complex data analysis tools.
 - *Human-friendly:* The simplicity of CSV files makes them easy for humans to read and understand, which is useful for quick data checks or manual edits.
- **Cons**

- *Large footprint*: CSV files can become quite large, especially with extensive datasets, necessitating compression to save space.
- *Parsing is CPU intensive*: Reading and processing CSV files can be demanding on the CPU, especially with large datasets.
- *No easy random access*: CSV files do not support efficient random access, making it difficult to quickly retrieve specific data points.
- *Can't read only a subset of columns*: You must read the entire file even if you only need a few columns.
- *No schema/types*: CSV files lack a built-in schema, meaning data types are not defined. Users often need to annotate CSV files with a schema for clarity.
- *Mainly read-only, hard to modify*: Modifying data within a CSV file can be cumbersome, as it often requires rewriting the entire file.

4 / 9: (Apache) Parquet

(Apache) Parquet

- **Parquet**
 - Reads data as tiles
 - Supports multi-dimensional, nested data
 - Generalizes dataframes
 - Column-storage
 - Stores each column together, uniform data type, compressed efficiently
 - IO layer executes queries
 - Reads only necessary data chunks from disk
- **Pros**
 - 10x smaller than CSV
 - 10x faster with multi-threading
 - Can read a subset of columns and rows
- **Cons**
 - Binary, not human-friendly
 - Requires an ingestion step to convert to Parquet
 - Mainly read-only, hard to modify



4 / 9

- **Apache Parquet**
 - *Reads data as tiles*: Parquet processes data in chunks, which helps in efficient data retrieval and processing.
 - *Supports multi-dimensional, nested data*: It can handle complex data structures, making it versatile for various data types beyond simple tables.
 - *Generalizes dataframes*: Parquet can manage data in a way that extends the capabilities of traditional dataframes, accommodating more complex data relationships.
 - *Column-storage*: Parquet stores data by columns rather than rows.
 - This means each column is stored together, allowing for efficient compression and faster query performance.
 - *IO layer executes queries*: The input/output layer is optimized to read only the neces-

sary data chunks, reducing the amount of data read from disk and speeding up query execution.

- **Pros**

- *10x smaller than CSV*: Parquet files are significantly more compact, saving storage space.
- *10x faster with multi-threading*: It can leverage multi-threading to speed up data processing.
- *Can read a subset of columns and rows*: Users can select specific data to read, improving efficiency.

- **Cons**

- *Binary, not human-friendly*: The data is stored in a binary format, making it difficult for humans to read directly.
- *Requires an ingestion step to convert to Parquet*: Data must be converted into Parquet format before use, adding an extra step.
- *Mainly read-only, hard to modify*: Parquet is optimized for reading data, and modifying existing data can be challenging.

5 / 9: JSON

JSON

- **JavaScript Object Notation**
- **Nested dictionaries and arrays**
- **Similar to XML**
 - More human-readable
 - Less boilerplate
 - Sometimes executable in JavaScript and Python

```
{
  "firstName": "John",
  "lastName": "Smith",
  "isAlive": true,
  "age": 25,
  "height_cm": 167.6,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    }
  ],
  "children": [],
  "spouse": null
}
```

5 / 9

- **JavaScript Object Notation (JSON)**

- JSON is a lightweight data interchange format that is easy for humans to read and write. It is also easy for machines to parse and generate. JSON is often used to transmit data between a server and a web application, serving as a bridge for data exchange.

- **Nested dictionaries and arrays**

- JSON supports complex data structures through the use of nested dictionaries (or objects) and arrays. This allows for the representation of hierarchical data, making it versatile for various applications.

- **Similar to XML**
 - JSON is often compared to XML, another data interchange format. However, JSON is generally more human-readable due to its simpler syntax. It requires less boilerplate code, which means there is less overhead in terms of additional tags and structure. JSON can sometimes be directly executed in languages like JavaScript and Python, making it a practical choice for developers.
- **Example JSON Structure**
 - The example provided shows a JSON object representing a person. It includes basic information such as name, age, and address. The address itself is a nested object, demonstrating JSON's ability to handle complex data. The phoneNumbers field is an array of objects, each representing a different phone number type. The children field is an empty array, indicating no children, and the spouse field is null, indicating no spouse. This example highlights JSON's flexibility in representing various data types and structures.

6 / 9: Protocol Buffers

Protocol Buffers

- **Open-source**
 - Developed by Google
- Represent data structures:
 - Language agnostic
 - Platform agnostic
 - Versioning
- **Schema is mostly relational**
 - Optional fields
 - Types
 - Default values
 - Structures
 - Arrays
- **Workflow**
 - Schema specified using a `.proto` file
 - Compiled by `protoc` to produce C++, Java, or Python code to initialize, read, serialize objects

```
import addressbook_pb2
person = addressbook_pb2.Person()
person.id = 1234
person.name = "John Doe"
person.email = "jdoe@example.com"
phone = person.phones.add()
phone.number = "555-4321"
phone.type = addressbook_pb2.Person.HOME

message Person {
  optional string name = 1;
  optional int32 id = 2;
  optional string email = 3;
  enum PhoneType {
    MOBILE = 0;
    HOME = 1;
    WORK = 2;
  }
  message PhoneNumber {
    optional string number = 1;
    optional PhoneType type = 2;
  }
  repeated PhoneNumber phones = 4;
}
```

6 / 9

- **Open-source**
 - Protocol Buffers, often abbreviated as Protobuf, is an open-source project developed by Google. This means that anyone can use, modify, and distribute it freely, which encourages widespread adoption and community contributions.
- **Represent data structures**
 - *Language agnostic*: Protobuf can be used with many programming languages, making it versatile for different development environments.
 - *Platform agnostic*: It works across various platforms, ensuring that data can be shared between systems regardless of their underlying architecture.

- *Versioning*: Protobuf supports versioning, allowing developers to evolve their data structures over time without breaking existing systems.
- **Schema is mostly relational**
 - *Optional fields*: Fields in Protobuf can be optional, providing flexibility in data representation.
 - *Types*: Protobuf supports various data types, ensuring precise data handling.
 - *Default values*: Fields can have default values, which are used if no explicit value is provided.
 - *Structures and Arrays*: Protobuf can define complex data structures, including arrays, to represent more intricate data models.
- **Workflow**
 - The schema for data is defined in a `.proto` file. This file outlines the structure and types of data.
 - The `protoc` compiler is used to generate code in languages like C++, Java, or Python. This code helps in initializing, reading, and serializing data objects, making it easier to work with structured data.

The Python code example shows how to use the generated classes to create and manipulate a `Person` object, while the `.proto` file defines the structure of the `Person` message, including fields like `name`, `id`, `email`, and a nested `PhoneNumber` message with an enumeration for phone types. This demonstrates how Protobuf can be used to define and work with structured data efficiently.

7 / 9: Serialization Formats

Serialization Formats

- **Avro**
 - Richer data structures
 - JSON-specified schema
- **Thrift**
 - Developed by Facebook
 - Now an Apache project
 - More languages supported
 - Supports exceptions and sets

```
{
  "namespace": "example.avro",
  "type": "record",
  "name": "User",
  "fields": [
    {
      "name": "name",
      "type": "string"
    },
    {
      "name": "favorite_number",
      "type": ["int", "null"]
    },
    {
      "name": "favorite_color",
      "type": ["string", "null"]
    }
  ]
}
```

7 / 9

– Serialization Formats

Serialization formats are essential for converting data structures or object states into a format that

can be stored or transmitted and then reconstructed later. Two popular serialization formats are Avro and Thrift.

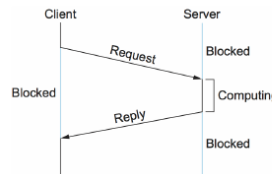
- **Avro**
 - Avro is known for supporting *richer data structures*, which means it can handle complex data types and nested data more effectively.
 - It uses a *JSON-specified schema*, which makes it easy to understand and work with. The schema defines the structure of the data, ensuring that both the data producer and consumer agree on the data format.
- **Thrift**
 - Thrift was originally *developed by Facebook* to facilitate scalable cross-language services development.
 - It is now an *Apache project*, which means it is maintained and improved by the open-source community.
 - Thrift supports *more programming languages*, making it versatile for different development environments.
 - It also supports *exceptions and sets*, providing additional functionality for handling errors and collections of unique items.

The JSON example on the right illustrates an Avro schema for a “User” record. It specifies fields like “name,” “favorite_number,” and “favorite_color,” with their respective data types. This schema ensures that any data serialized with Avro adheres to this structure, promoting consistency and reliability in data handling.

8 / 9: Remote Procedure Call

Remote Procedure Call

- **Remote Procedure Call (RPC)**
 - Requests services from programs on other computers
 - Abstracts network communication
- **Goal:** Make remote calls like local procedure calls without network details
 - Used in distributed systems
 - E.g., microservices, cloud services, client-server applications
 - Can be synchronous or asynchronous
- **Problems**
 - Can't serialize pointers
 - Handling asynchronous communication
 - Failures and retries



8 / 9

- **Remote Procedure Call (RPC)**

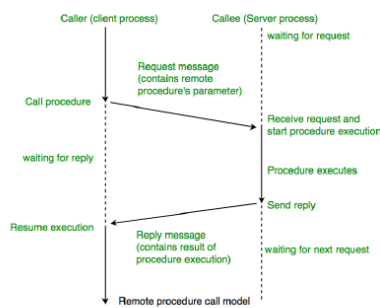
- RPC is a protocol that allows a program to request a service from a program located on another computer in a network. It simplifies the process of network communication by abstracting the complexities involved.
- **Goal**
 - The main aim of RPC is to make remote calls appear as if they are local procedure calls. This means developers can focus on the logic of their applications without worrying about the underlying network details.
 - RPC is widely used in distributed systems, such as microservices, cloud services, and client-server applications. These systems often require components to communicate across different networked environments.
 - RPC can operate in two modes: synchronous, where the client waits for the server to respond, and asynchronous, where the client continues processing other tasks while waiting for the server's response.
- **Problems**
 - One challenge with RPC is that it cannot serialize pointers, which are memory addresses used in programming. This limitation requires careful handling of data structures.
 - Managing asynchronous communication can be complex, as it involves ensuring that messages are correctly sent and received without blocking the system.
 - Handling failures and implementing retries are critical, as network issues can cause requests to fail. Proper error handling and retry mechanisms are necessary to ensure reliability.

The diagram illustrates a typical RPC interaction where the client sends a request to the server, both are blocked during processing, and then the server sends a reply back to the client.

9 / 9: RPCs: Internals

RPCs: Internals

- *Client procedure call*: Client calls stub function with arguments
- *Request marshalling*: Client stub serializes arguments for network transmission
- *Server communication*: Client's RPC runtime sends request to server
- *Server-side unmarshalling*: Server's RPC runtime deserializes arguments
- *Procedure execution*: Server calls procedure
- *Response marshalling*: Return values marshaled into response message
- *Client communication / response unmarshalling / return to client*: Return values passed back to client's stub call, execution continues locally



- **Client procedure call:** The process begins when the client calls a stub function, which acts as a placeholder for the actual remote procedure. The client provides the necessary arguments for the procedure.
- **Request marshalling:** The client stub takes these arguments and serializes them. Serialization is the process of converting data into a format that can be easily transmitted over a network.
- **Server communication:** The client's RPC (Remote Procedure Call) runtime sends the serialized request to the server. This involves network communication to ensure the request reaches the correct destination.
- **Server-side unmarshalling:** Upon receiving the request, the server's RPC runtime deserializes the arguments. This means converting the serialized data back into a format that the server can understand and use.
- **Procedure execution:** The server then calls the actual procedure using the deserialized arguments. This is where the main logic of the remote procedure is executed.
- **Response marshalling:** After the procedure is executed, the server marshals the return values. This involves serializing the results into a response message that can be sent back to the client.
- **Client communication / response unmarshalling / return to client:** The response message is sent back to the client. The client's RPC runtime then deserializes the return values, and the execution continues locally as if the procedure was executed on the client side. This seamless integration is a key feature of RPCs, making remote calls appear like local ones.